

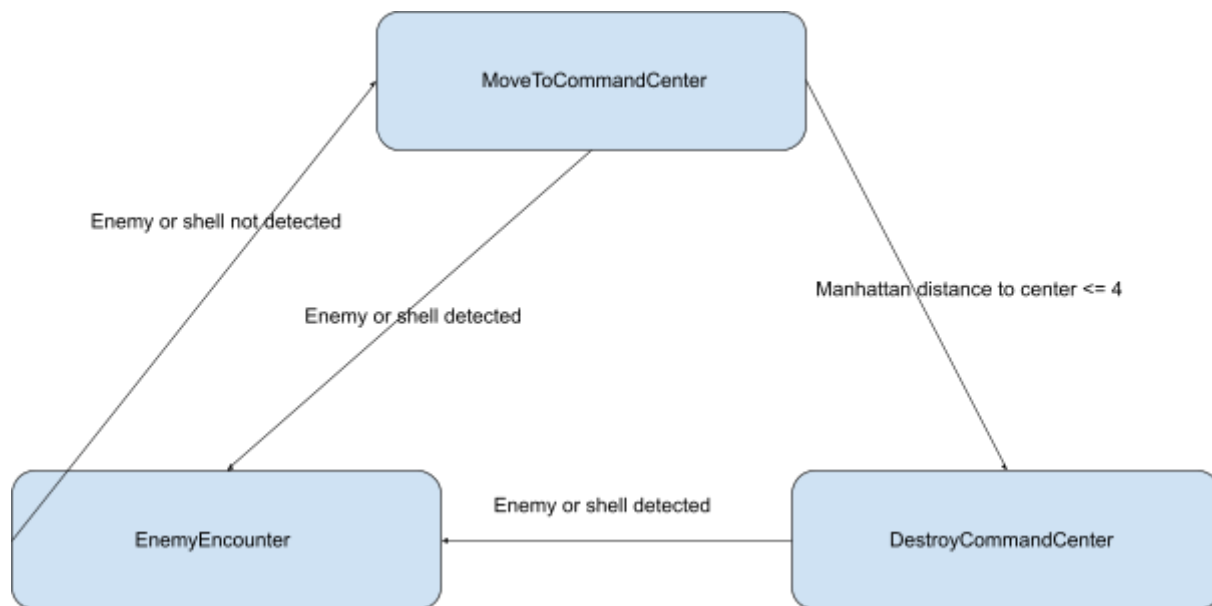
Memoria practica 02: Agentes

Jaime Sánchez Izquierdo

Juan Girón Herranz

1. Máquina de estados.....	1
2. Descripción del entorno usando REAS.....	1
2. Tipo de Agente.....	2
3. Diseño Preliminar del Agente.....	2
4. Diseño Final del Agente.....	2

1. Máquina de estados



2. Descripción del Entorno usando REAS

R (Recursos): El entorno del videojuego **Battle City** está compuesto por un mapa con diferentes tipos de elementos, como paredes de **ladrillo** (rompibles), **paredes metálicas** (indestructibles) y pasillos. Existen **dos agentes enemigos** (uno aleatorio y otro reactivo) y un **jugador**, además de una **Command Center** que debe ser defendida de los agentes. Todos los tanques son capaces de moverse en cuatro direcciones y disparar.

E (Entidades): En el entorno existen varias entidades: el agente reactivo, el agente aleatorio, el jugador, el Command Center, Shell(balas) en movimiento, paredes de ladrillo y paredes metálicas.

A (Atributos): Cada entidad tiene atributos específicos. El agente tiene una posición en el mapa, un estado de salud (dos puntos de vida), la capacidad de moverse y disparar. El jugador tiene un punto de vida y el Command Center puede recibir disparos hasta ser destruido. Las balas se destruyen al chocar con las paredes o con otras balas.

S (Sensores): El agente recibe información sobre los elementos en su entorno inmediato, incluyendo la presencia de obstáculos, la posición(eje x e y) del jugador y del Command Center, y la posibilidad de disparar(si su bala no se encuentra en el mapa en ese momento). La percepción se da a través de un vector con valores flotantes llamado perception.

2. Tipo de Agente

El agente desarrollado es un **agente reactivo basado en máquina de estados finitos (FSM, Finite State Machine)**. No emplea planificación avanzada ni aprendizaje, sino que responde directamente a su entorno según las reglas predefinidas de sus estados. Tiene tres estados principales:

- **MoveToCommandCenter:** Se desplaza hacia la Command Center evitando obstáculos.
- **EnemyEncounter:** Se encarga de atacar a enemigos cuando los detecta.
- **DestroyCommandCenter:** Dispara a la Command Center cuando está en rango.

3. Diseño Preliminar del Agente

Inicialmente, el diseño del agente se planteó de la siguiente manera:

- Definir un comportamiento básico de movimiento hacia el **Command Center**.
- Implementar un sistema de evasión de obstáculos.
- Programar la detección y ataque a enemigos.
- Priorizar la destrucción del **Command Center** cuando sea viable.

En esta etapa, se consideró la posibilidad de agregar una estrategia más compleja para mejorar la eficiencia del agente, como el uso de algoritmos de pathfinding o evasión inteligente.

4. Diseño del Agente

Tests del agente pasados con **snapshot por defecto 0,20**.

Hemos implementado el diseño mediante con los siguientes estados:

MoveToCommandCenter

- Update

Primero calcula la distancia del agente al center en cada eje (deltaX, deltaY).

El agente intenta realizar en estas acciones en orden con las siguientes prioridades:

- 1- Moverse hacia el center en el eje con más delta si no hay obstáculo.
- 2- Moverse hacia el center en el eje con menos delta si no hay obstáculo.
- 3- Romper el obstáculo si es rompible en el eje con más delta.
- 4- Romper el obstáculo si es rompible en el eje con menos delta.
- 5- Movimiento aleatorio, si hay obstáculo rompible lo rompe.

- Transit

Si detecta un enemigo o una shell pasamos al estado **EnemyEncounter**.

Si la distancia manhattan al center es ≤ 4 pasamos a **DestroyCommandCenter**.

En otro caso no cambiamos de estado.

EnemyEncounter

- Update

Si el agente puede disparar, disparará hacia la shell o player detectado.

Si no puede disparar, esquivará hacia una dirección sin obstáculos.

- Transit

Si detecta un enemigo o una shell nos quedamos en **EnemyEncounter**.

En otro caso vuelve a **MoveToCommandCenter**.

DestroyCommandCenter

- Update

Nos movemos y disparamos en el eje con más distancia al center que no tenga un obstáculo indestructible.

- Transit

Si detectamos un enemigo o una shell pasamos a **EnemyEncounter**.

En otro caso nos quedamos en **DestroyCommandCenter**.

